

Carpeta de Campo - MiniProyecto de Gestión de FarmaCity

Laboratorio de algoritmos y estructura de datos con base de datos

Alumna: Alba Spitalnik

Curso: 4º4

Fecha de inicio: 7/12

Profesor: Ignacio Traverso

Periodo de recuperacion diciembre

Código de la base de datos:

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Servidor: 127.0.0.1
-- Tiempo de generación: 07-12-2025 a las 22:19:18
-- Versión del servidor: 10.4.32-MariaDB
-- Versión de PHP: 8.2.12

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Base de datos: `farmacity`
--
```

```

-- -----
--
-- Estructura de tabla para la tabla `medicamentos`
--

CREATE TABLE `medicamentos` (
  `ID` int(10) UNSIGNED NOT NULL,
  `nombre` varchar(100) NOT NULL,
  `categoria` varchar(50) DEFAULT NULL,
  `precio` decimal(10,2) NOT NULL,
  `stock` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Volcado de datos para la tabla `medicamentos`
--

INSERT INTO `medicamentos` (`ID`, `nombre`, `categoria`, `precio`,
`stock`) VALUES
(1, 'Paracetamol 500mg', 'Analgésico', 2.50, 100),
(2, 'Ibuprofeno 400mg', 'Antiinflamatorio', 3.75, 80),
(3, 'Amoxicilina 500mg', 'Antibiótico', 5.90, 50);

-- -----
--
-- Estructura de tabla para la tabla `ventas`
--

CREATE TABLE `ventas` (
  `ID` int(10) UNSIGNED NOT NULL,
  `ID_Medicamento` int(10) UNSIGNED NOT NULL,
  `fecha` date NOT NULL,
  `cantidad` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Volcado de datos para la tabla `ventas`
--

INSERT INTO `ventas` (`ID`, `ID_Medicamento`, `fecha`, `cantidad`)
VALUES

```

```
(1, 1, '2025-12-01', 5),
(2, 2, '2025-12-02', 3),
(3, 1, '2025-12-03', 2),
(4, 3, '2025-12-03', 1);

--
-- Índices para tablas volcadas
--
--
-- Indices de la tabla `medicamentos`
--
ALTER TABLE `medicamentos`
  ADD PRIMARY KEY (`ID`);

--
-- Indices de la tabla `ventas`
--
ALTER TABLE `ventas`
  ADD PRIMARY KEY (`ID`),
  ADD KEY `ID_Medicamento` (`ID_Medicamento`);

--
-- AUTO_INCREMENT de las tablas volcadas
--
--
-- AUTO_INCREMENT de la tabla `medicamentos`
--
ALTER TABLE `medicamentos`
  MODIFY `ID` int(10) UNSIGNED NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=4;

--
-- AUTO_INCREMENT de la tabla `ventas`
--
ALTER TABLE `ventas`
  MODIFY `ID` int(10) UNSIGNED NOT NULL AUTO_INCREMENT,
  AUTO_INCREMENT=5;

--
-- Restricciones para tablas volcadas
--
```

```
--
-- Filtros para la tabla `ventas`
--
ALTER TABLE `ventas`
  ADD CONSTRAINT `ventas_ibfk_1` FOREIGN KEY (`ID_Medicamento`)
REFERENCES `medicamentos` (`ID`);
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```

Código de la conexión de la base de datos en python y las funciones pedidas:

```
import mysql.connector
from mysql.connector import
errorcode
import datetime
import json

cursor = None
cnx = None

def conectarBase():
    global cnx, cursor
```

```
try:
    cnx =
mysql.connector.connect(user="root"
, password="", host="Localhost",
database="farmacity")
    cursor =
cnx.cursor(dictionary=True)
    print('Conexión
establecida')

except mysql.connector.Error as
err:
    if err.errno ==
errorcode.ER_ACCESS_DENIED_ERROR:
        print('Usuario o
contraseña incorrectos!')
    elif err.errno ==
errorcode.ER_BAD_DB_ERROR:
        print('La base de datos
no existe!')
    else:
        print(err)

conectarBase()
```

```
def consultaSelect(tabla):
    Consulta = f"SELECT * FROM
{tabla};"
    cursor.execute(Consulta)
    return cursor.fetchall()

def ordenarProductos():
    medicamentos =
consultaSelect("medicamentos")
    lista = medicamentos
    n = len(lista)
    for i in range(n - 1):
        hay_cambio = False
        for j in range(0, n - i -
1):
            if lista[j]["ID"] >
lista[j + 1]["ID"]:
                lista[j], lista[j +
1] = lista[j + 1], lista[j]
                Hay_Cambio = True
        if not hay_cambio:
```

```
        break
    return lista

def busquedaProducto(ID):
    arreglo = ordenarProductos()
    valor = ID
    izquierda = 0
    derecha = len(arreglo) - 1
    while izquierda <= derecha:
        medio = (izquierda +
derecha) // 2
        if arreglo[medio]["ID"] ==
valor:
            return arreglo[medio]
        elif arreglo[medio]["ID"] <
valor:
            izquierda = medio + 1
        else:
            derecha = medio - 1
    return -1

def vender():
```

```
    producto =  
busquedaProducto(int(input("Ingrese  
el ID del producto: ")))  
    ID_producto = producto["ID"]  
    stock = producto["stock"]  
    cantidad = int(input("Ingrese la  
cantidad que desea vender: "))  
  
    if cantidad > stock:  
        print(f"Stock insuficiente.  
Solo hay {stock} unidades  
disponibles.")  
        return  
    fecha_compra =  
datetime.datetime.now().strftime("%  
Y-%m-%d")  
    sql = "INSERT INTO ventas  
(ID_Medicamento, fecha, cantidad)  
VALUES (%s, %s, %s)"  
    cursor.execute(sql,  
(ID_producto, fecha_compra,  
cantidad))  
  
    nuevo_stock = stock - cantidad
```

```
sql_update = "UPDATE
medicamentos SET stock = %s WHERE
ID = %s"

cursor.execute(sql_update,
(nuevo_stock, ID_producto))

cnx.commit()

print(f"Compra registrada con
éxito.")

def agregar_medicamento():
    nombre = input("Ingrese el
nombre del medicamento: ")
    categoria = input("Ingrese el
categoria del medicamento: ")
    precio = float(input("Ingrese el
precio del medicamento: "))
    stock = int(input("Ingrese el
stock del medicamento: "))
    sql = "INSERT INTO
medicamentos(nombre, categoria,
precio, stock) VALUES (%s, %s, %s,
%s) "
```

```
        cursor.execute(sql, (nombre,
categoria, precio, stock))
        cnx.commit()

def mas_vendidos():
    consulta = "Select m.id,
m.nombre, v.cantidad FROM
medicamentos m join ventas v on
v.ID_Medicamento = m.id order by
v.cantidad desc LIMIT 5";
    cursor.execute(consulta)
    cnx.commit()
    ParaJson =
str(cursor.fetchall())
    with open("mas_vendidos.json",
'w', encoding='utf-8') as archivo:
        json.dump(ParaJson, archivo,
indent=4, ensure_ascii=False)
    return cursor.fetchall()

def stock_critico():
    consulta = "Select * from
medicamentos where stock < 10"
    cursor.execute(consulta)
    cnx.commit()
```

```
        ParaJson =
str(cursor.fetchall())
        with open("stock_critico.json",
'w', encoding='utf-8') as archivo:
            json.dump(ParaJson, archivo,
indent=4, ensure_ascii=False)
        return cursor.fetchall()

def compras_5_mes():
    mes = int(input("Ingrese el mes:
"))
    consulta = f"SELECT * from
ventas where fecha like '%%{mes}%%'
and cantidad > 5"
    cursor.execute(consulta)
    cnx.commit()
    ParaJson =
str(cursor.fetchall())
    with open("stock_critico.json",
'w', encoding='utf-8') as archivo:
        json.dump(ParaJson, archivo,
indent=4, ensure_ascii=False)
    return cursor.fetchall()

def eliminar_medimento():
```

```
    ID_medicamento =
int(input("Ingrese el ID del
medicamento: "))
    borrar = f"DELETE FROM
medicamentos WHERE ID =
{ID_medicamento}"
    cursor.execute(borrar)
    cnx.commit()

def borrar_ventas():
    ID_venta = int(input("Ingrese el
ID de la venta: "))
    borrar = f"DELETE FROM ventas
WHERE ID = {ID_venta}"
    cursor.execute(borrar)
    cnx.commit()

def mostrar_medicamento_por_ID():
    producto =
busquedaProducto(int(input("Ingrese
el ID del producto: ")))
    print(producto)

def mostrar_venta_por_ID():
```

```
    producto =  
busquedaProducto(int(input("Ingrese  
el ID de la venta: ")))  
    print(producto)  
  
def mostrar_tablas():  
    consulta_medicamentos =  
consultaSelect("medicamentos")  
    consulta_ventas =  
consultaSelect("ventas")  
    print(consulta_medicamentos)  
    print(consulta_ventas)  
    contador = 0  
    for x in consulta_ventas:  
        for y in  
consulta_medicamentos:  
            if x["ID_Medicamento"]  
== y["ID"]:  
                contador +=  
y["precio"]  
    print(contador)
```